UNIVERSIDADE SÃO JUDAS TADEU
CURSO DE ENGENHARIA DE SOFTWARE
UNIDADE CURRICULAR SISTEMAS AUTOMATIZADOS

MARCELO ANTONIO PEREIRA MARCOLINO

Versão preliminar — integrantes do grupo a definir

SEMÁFORO INTELIGENTE

controle de um cruzamento com fase de pedestres em CLP simulado

São Paulo

2026

MARCELO ANTONIO PEREIRA MARCOLINO

Versão preliminar — integrantes do grupo a definir

SEMÁFORO INTELIGENTE

controle de um cruzamento com fase de pedestres em CLP simulado

Relatório técnico apresentado à unidade curricular Sistemas Automatizados do curso de Engenharia de Software da Universidade São Judas Tadeu, turma ESO1AN-MCE3, como projeto da avaliação A3 do semestre 2026.2.

Professor: Prof. Robson Calvetti

São Paulo

2026

RESUMO

Este relatório apresenta o projeto, a implementação e a verificação de um semáforo para o cruzamento de uma via principal com uma via secundária, com travessia de pedestres em fase exclusiva, programado em linguagem Ladder no PLC Simulator Online. O controlador é atuado por demanda: sem veículos na via secundária nem pedidos de pedestre, a via principal permanece verde; um laço detector conta a fila da via secundária, e três ou mais veículos encerram o verde principal assim que o tempo mínimo é cumprido, enquanto um veículo isolado aguarda o tempo máximo. O pedido de pedestre é memorizado e sinalizado, há intervalos de amarelo e de vermelho geral entre fluxos e um modo de amarelo intermitente para manutenção. O comportamento foi modelado em GRAFCET com nove etapas e convertido para Ladder em 55 rungs, com as transições calculadas antes da atualização das etapas e cada lâmpada escrita em uma única bobina. A verificação executou 90 casos de teste no motor do próprio simulador, com o resultado esperado escrito antes da execução, todos aprovados; um interpretador independente reproduziu os 32 683 registros de varredura; oito invariantes de segurança não foram violados nas condições testadas e seis versões do programa com defeitos propositais foram detectadas pelos testes. Os resultados indicam que o programa cumpre a sequência especificada dentro das condições ensaiadas, com as limitações de uma simulação lógica discutidas no texto.

Palavras-chave: controlador lógico programável; Ladder; GRAFCET; semáforo atuado; simulação.

ABSTRACT

This report presents the design, implementation and verification of a traffic light for the intersection of a main road and a secondary road, with an exclusive pedestrian phase, programmed in ladder logic on PLC Simulator Online. The controller is demand-actuated: with no vehicles on the secondary road and no pedestrian requests, the main road stays green; an inductive loop counts the queue on the secondary road, and three or more vehicles end the main green as soon as its minimum time is met, while a single vehicle waits for the maximum time. Pedestrian requests are latched and signaled, amber and all-red intervals separate conflicting flows, and a flashing-amber mode supports maintenance. The behavior was modeled as a GRAFCET with nine steps and converted into 55 ladder rungs, with transitions computed before the steps are updated and each lamp written by a single coil. Verification ran 90 test cases on the simulator engine itself, with expected results written before execution, all of which passed; an independent interpreter reproduced all 32,683 scan records; eight safety invariants were not violated under the tested conditions, and six deliberately faulty versions of the program were detected by the tests. The results indicate that the program performs the specified sequence under the tested conditions, within the limits of a logical simulation discussed in the text.

Keywords: programmable logic controller; ladder logic; GRAFCET; actuated traffic signal; simulation.

SUMÁRIO

1 INTRODUÇÃO 5
2 FUNDAMENTAÇÃO TEÓRICA 6
2.1 Sistemas automatizados e o controlador lógico programável 6
2.2 Linguagem Ladder 6
2.3 GRAFCET e conversão para Ladder 7
2.4 Simulação e verificação 8
2.5 Controle de cruzamentos semaforizados 8
3 DESCRIÇÃO DO SISTEMA AUTOMATIZADO 9
3.1 Problema e requisitos 9
3.2 Sensores e atuadores 9
3.3 Arquitetura da automação 10
4 MODELAGEM DO SISTEMA 11
4.1 Etapas e ações 11
4.2 Transições e tempos 12
5 DESCRIÇÃO DA LÓGICA DE CONTROLE 13
5.1 Organização do programa 13
5.2 Fila da via B e pedido de pedestre 13
5.3 Transições, etapas e saídas 14
5.4 Decisões de projeto 14
6 RESULTADOS DA SIMULAÇÃO 15
6.1 Casos de teste 15
6.2 Invariantes e mutantes 15
6.3 Estados observados no simulador 16
7 DISCUSSÃO DOS RESULTADOS 17
8 CONCLUSÃO 18
REFERÊNCIAS 19

1 INTRODUÇÃO

Cruzamentos controlados por semáforo são um exemplo cotidiano de sistema automatizado: sensores informam a presença de veículos e pedestres, um controlador decide a sequência dos sinais e as lâmpadas comunicam a decisão aos usuários da via. O projeto da avaliação A3 da unidade curricular Sistemas Automatizados propõe desenvolver e simular um sistema desse tipo com lógica de controlador lógico programável (CLP), identificando sensores e atuadores, construindo a lógica sequencial e documentando os resultados (Calvetti, 2026e).

Este trabalho trata do tema do semáforo inteligente. O objetivo geral é projetar, implementar e verificar, em simulador de CLP, o controle de um cruzamento entre uma via principal (A) e uma via secundária (B), com travessia de pedestres. Os objetivos específicos são:

- identificar sensores e atuadores e definir a política de controle, incluindo a segurança entre fluxos que se cruzam;
- modelar a sequência em GRAFCET e convertê-la para Ladder no PLC Simulator Online;
- usar temporizadores e contadores de forma funcional — tempos de cada fase, verde mínimo e máximo, contagem da fila;
- verificar o programa com casos de teste, invariantes de segurança e mutantes, e publicar o programa para demonstração.

O capítulo 2 resume os conceitos usados; o capítulo 3 descreve o sistema; o capítulo 4 apresenta a modelagem; o capítulo 5, a lógica de controle; o capítulo 6, os resultados da simulação; o capítulo 7 discute esses resultados e suas limitações; o capítulo 8 conclui o trabalho.

2 FUNDAMENTAÇÃO TEÓRICA

2.1 Sistemas automatizados e o controlador lógico programável

Um sistema automatizado organiza observação, decisão e ação para que um processo cumpra seu objetivo de forma repetível e segura. Sensores informam o estado do processo, o controlador aplica regras sobre essas informações e os atuadores produzem o efeito físico; o resultado da ação volta a ser percebido pelos sensores, o que fecha um ciclo de realimentação (Calvetti, 2026b). Neste projeto todos os sensores são discretos — presença ou ausência, nível acima ou abaixo de um ponto, posição atingida ou não — e chegam ao controlador como sinais booleanos (Calvetti, 2026c).

O controlador lógico programável (CLP) executa o programa em ciclos repetidos, chamados de varredura: lê as entradas, avalia a lógica e atualiza as saídas. Como a leitura acontece em um instante do ciclo, uma entrada que muda logo depois dela só é percebida na varredura seguinte (Calvetti, 2026a). Essa característica tem duas consequências práticas que aparecem ao longo deste trabalho: no simulador usado, os temporizadores avançam em passos de uma varredura, e a ordem em que as regras são avaliadas dentro do ciclo influencia o resultado.

2.2 Linguagem Ladder

A Ladder é uma das linguagens de programação de CLPs relacionadas à IEC 61131-3 e tem leitura próxima à dos antigos circuitos de relés (Calvetti, 2026a). Cada linha do diagrama, o rung, representa uma continuidade lógica entre o trilho da esquerda e um elemento de saída: contatos em série realizam a função E, caminhos em paralelo realizam a função OU e um contato negado realiza a função NÃO. Um contato da própria saída em paralelo com o comando de partida cria o selo, que mantém a saída ligada depois que o botão é solto. Escrever a mesma saída em mais de uma bobina torna o comportamento difícil de prever (Calvetti, 2026a); no simulador usado, prevalece a última bobina avaliada na varredura.

Além da bobina comum, que acompanha a condição do rung a cada varredura, há bobinas de memória (SET e RESET), que ligam ou desligam um bit e o mantêm até um comando oposto (Calvetti, 2026d). Os temporizadores medem o tempo em que

uma condição permanece verdadeira: o TON liga a saída depois que a entrada fica ativa durante o tempo predefinido (PT), com o tempo decorrido (ET) disponível para leitura; o TOF mantém a saída ligada durante PT depois que a entrada cai. Os contadores reagem a bordas: o CTU soma uma unidade a cada transição de 0 para 1 na entrada, e sua saída indica quando o valor corrente (CV) alcança ou supera o predefinido (PV); o CTD subtrai uma unidade a cada borda; a política entre zeragem e contagem simultâneas precisa ser definida e testada (Calvetti, 2026d). No simulador usado, a saída do CTD (QD) indica CV ≤ 0, e uma entrada de carga copia PV para CV (CodingPLC, 2026).

O simulador usado oferece também o temporizador retentivo (TONR), que conserva o tempo acumulado quando a entrada cai e só zera por um comando próprio, o contador bidirecional (CTUD) e contatos que conduzem apenas na borda de subida ou de descida de um sinal (CodingPLC, 2026). A detecção de borda faz um comando valer uma única vez, mesmo com o botão mantido pressionado.

2.3 GRAFCET e conversão para Ladder

O GRAFCET, definido pela IEC 60848, descreve o comportamento de um sistema sequencial por etapas, transições e ações (Calvetti, 2026f). Uma etapa representa uma situação do sistema e pode estar ativa ou inativa; a etapa inicial define a situação depois da inicialização. Cada transição liga etapas consecutivas e tem uma receptividade, a condição lógica que permite a mudança. A transição só dispara quando está habilitada — suas etapas anteriores estão ativas — e a receptividade é verdadeira; o disparo desativa as etapas anteriores e ativa as seguintes. As ações descrevem o que o sistema faz enquanto cada etapa está ativa (Calvetti, 2026f).

A conversão adotada para Ladder segue o padrão trabalhado na disciplina para programação sequencial (Calvetti, 2026d), organizado em blocos na seguinte ordem dentro da varredura: primeiro todas as transições são calculadas como bits, a partir das etapas ativas no início do ciclo; depois cada transição disparada desliga a etapa de origem e liga a de destino por bobinas SET e RESET; em seguida vêm os temporizadores e contadores; por fim as saídas, cada uma escrita em uma única bobina. Calcular as transições antes de atualizar as etapas impede que a sequência avance mais de um passo na mesma varredura.

2.4 Simulação e verificação

O programa foi desenvolvido e executado no PLC Simulator Online, um editor e simulador de Ladder que funciona no navegador (CodingPLC, 2026). Neste simulador, cada varredura corresponde a 66 ms e o tempo decorrido avança 66 ms por varredura, de modo que um temporizador com PT de 1 s conclui na décima sexta varredura; em um CLP real, a base de tempo depende do equipamento. A verificação combinou quatro recursos:

- casos de teste com o resultado esperado escrito antes da execução e executados no motor do simulador, varredura a varredura;
- um interpretador independente, que executa o mesmo programa e precisa produzir os mesmos registros de varredura (todas as tags observadas depois de cada varredura);
- invariantes de segurança, vigiados em toda varredura com a simulação ligada;
- teste de mutação: versões do programa com um defeito proposital (mutantes) precisam ser reprovadas pelos casos, o que mostra que os testes detectam aquele tipo de erro (Jia; Harman, 2011).

Antes de serem usados, os blocos TONR, CTD, CTUD, os contatos de borda e as operações aritméticas foram caracterizados em 68 casos próprios no motor do simulador, todos aprovados.

2.5 Controle de cruzamentos semaforizados

Num cruzamento, dois fluxos são conflitantes quando suas trajetórias se cruzam, e o requisito central de segurança é que eles nunca recebam direito de passagem ao mesmo tempo; entre o verde de um e o do outro, o amarelo avisa o fim da passagem e o vermelho geral protege quem ainda está no cruzamento. Um semáforo de tempo fixo repete o mesmo ciclo; um semáforo atuado usa detectores, como laços e botoeiras, para só servir um fluxo quando há demanda, respeitando tempos mínimos e máximos de verde. Os tempos deste projeto foram reduzidos para a demonstração, e as ordens de grandeza reais citadas são apenas indicativas, sem consulta a norma de trânsito.

3 DESCRIÇÃO DO SISTEMA AUTOMATIZADO

3.1 Problema e requisitos

O cruzamento tem uma via principal A, de maior tráfego, e uma via secundária B. A travessia de pedestres é feita em fase exclusiva: durante ela, todos os veículos ficam parados. O controlador deve:

- manter a via A verde enquanto não houver demanda em B nem pedido de pedestre;
- atender a via B quando houver veículos esperando, mais cedo quando a fila for grande;
- registrar o pedido de pedestre com um único toque no botão e sinalizá-lo até a travessia começar;
- separar fluxos conflitantes por amarelo e vermelho geral e nunca abrir dois fluxos conflitantes juntos;
- oferecer um modo de amarelo intermitente para manutenção ou falha, retornando sempre pelo vermelho geral.

3.2 Sensores e atuadores

O Quadro 1 lista as entradas e saídas do CLP. Todas são discretas; os grupos focais têm uma saída por lâmpada.

Quadro 1 – Entradas e saídas do controlador

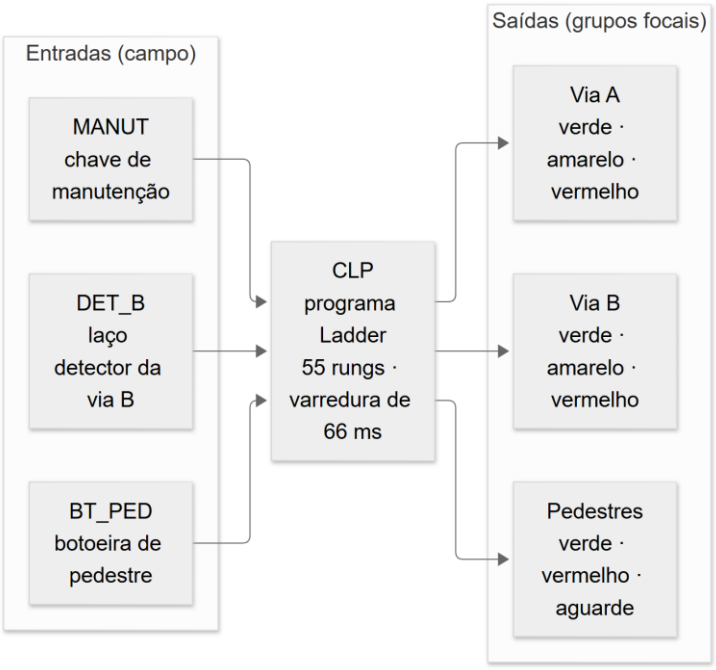
Tag	Tipo	Função
`DET_B`	entrada	laço detector na faixa de retenção da via B (1 = veículo sobre o laço)
`BT_PED`	entrada	botoeira de pedestre (1 enquanto pressionada)
`MANUT`	entrada	chave de manutenção: 1 = amarelo intermitente
`A_VERDE`, `A_AMARELO`, `A_VERMELHO`	saídas	grupo focal da via A
`B_VERDE`, `B_AMARELO`, `B_VERMELHO`	saídas	grupo focal da via B
`PED_VERDE`, `PED_VERMELHO`	saídas	grupo focal de pedestres
`PED_AGUARDE`	saída	lâmpada "aguarde" na botoeira (pedido registrado)

Fonte: elaborado pelo autor (2026).

3.3 Arquitetura da automação

A Figura 1 mostra a arquitetura: três sinais de campo entram no CLP, que executa o programa Ladder a cada varredura de 66 ms e comanda os três grupos focais. No simulador, as entradas são acionadas clicando nos contatos e as saídas aparecem na tabela de tags e no próprio diagrama. O programa está publicado e abre sem login em https://app.plcsimulator.online/wuPLOb3XvySEne6i8krQ.

Figura 1 – Arquitetura da automação do semáforo

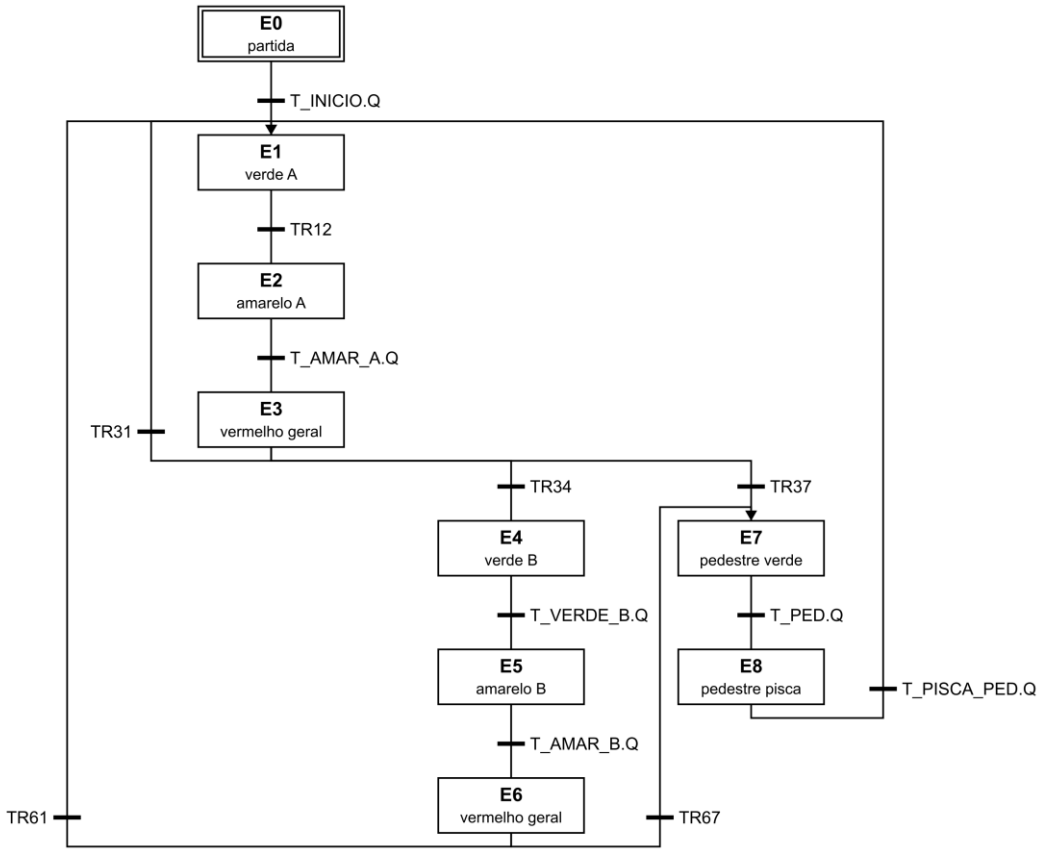


Fonte: elaborada pelo autor (2026).

4 MODELAGEM DO SISTEMA

O comportamento foi modelado em GRAFCET com nove etapas (Figura 2). A etapa E0 é a partida, com tudo vermelho; E1 é o repouso em verde na via A; E2 e E3 encerram o verde A com amarelo e vermelho geral; E4 a E6 servem a via B; E7 e E8 servem os pedestres, com o vermelho de pedestre piscando no fim da travessia. A chave `MANUT` fica fora do GRAFCET: enquanto vale 1, zera todas as etapas e liga o amarelo intermitente; ao voltar a 0, o programa recomeça por E0, de modo que a saída da manutenção sempre passa pelo vermelho geral.

Figura 2 – GRAFCET do semáforo (etapas, transições e tempos de demonstração)



Fonte: elaborada pelo autor (2026).

4.1 Etapas e ações

Quadro 2 – Lâmpadas acesas em cada etapa

Etapa	Via A	Via B	Pedestres
E0, E3, E6	vermelho	vermelho	vermelho
E1	verde	vermelho	vermelho

Etapa	Via A	Via B	Pedestres
E2	amarelo	vermelho	vermelho
E4	vermelho	verde	vermelho
E5	vermelho	amarelo	vermelho
E7	vermelho	vermelho	verde
E8	vermelho	vermelho	vermelho piscando
manutenção	amarelo piscando	amarelo piscando	apagado

Fonte: elaborado pelo autor (2026).

4.2 Transições e tempos

As transições sem tempo próprio estão no Quadro 3; as demais terminam quando o temporizador da etapa conclui. Depois do vermelho geral, a via B tem prioridade e o pedestre é servido em seguida, sem voltar ao verde A.

Quadro 3 – Transições condicionais e suas receptividades

Bit	Etapas	Receptividade
`TR12`	E1 → E2	verde mínimo cumprido e (pedido de pedestre ou fila ≥ 3 ou demanda em B com verde máximo cumprido)
`TR34`	E3 → E4	vermelho geral cumprido e demanda em B
`TR37`	E3 → E7	vermelho geral cumprido, sem demanda em B, com pedido de pedestre
`TR31`	E3 → E1	vermelho geral cumprido, sem demanda nem pedido
`TR67`	E6 → E7	vermelho geral cumprido e pedido de pedestre
`TR61`	E6 → E1	vermelho geral cumprido, sem pedido

Fonte: elaborado pelo autor (2026).

O Quadro 4 resume os tempos, reduzidos para a demonstração e convertidos em varreduras pela regra do simulador (⌈PT/66 ms⌉).

Quadro 4 – Tempos de demonstração e sua duração em varreduras

Temporizador	PT	Varreduras	Ordem de grandeza real
`T_INICIO` vermelho geral na partida	2 s	31	alguns segundos
`T_MIN_A` verde mínimo de A	8 s	122	dezenas de segundos
`T_MAX_A` verde máximo de A	16 s	243	um a dois minutos
`T_AMAR_A`, `T_AMAR_B` amarelo	3 s	46	alguns segundos
`T_TV_A`, `T_TV_B` vermelho geral	1 s	16	um a três segundos
`T_VERDE_B` verde de B	6 s	91	dezenas de segundos
`T_PED` verde de pedestre	6 s	91	dezenas de segundos
`T_PISCA_PED` vermelho piscante	4 s	61	alguns segundos

Fonte: elaborado pelo autor (2026).

5 DESCRIÇÃO DA LÓGICA DE CONTROLE

5.1 Organização do programa

O programa tem 55 rungs, 180 elementos e 121 tags (contando as sub-tags de temporizadores e do contador) e segue a ordem descrita no capítulo 2 (Quadro 5). Cada rung tem um comentário com seu código e sua função, gravado no próprio arquivo do projeto.

Quadro 5 – Blocos do programa Ladder

Bloco	Rungs	O que faz
Fila e pedidos	F1–F4	zera a fila quando B abre; conta a fila; calcula a demanda; memoriza o pedido de pedestre
Transições	T01–T81	doze bits de transição, calculados a partir das etapas do início da varredura
Disparo	S01–S81, SP, MN	desliga a etapa de origem e liga a de destino; atende o pedido; zera tudo em manutenção
Temporizadores	TM0–TM9	um TON por etapa temporizada (E1 tem dois: mínimo e máximo)
Intermitência	P1–P5	gerador de 0,5 s aceso / 0,5 s apagado, com a fase guardada em memória
Lâmpadas	O1–O9	uma bobina por lâmpada; vermelho escrito como “nem verde nem amarelo”
Partida	IN (último)	liga E0 e a memória de partida ao carregar o projeto ou sair da manutenção

Fonte: elaborado pelo autor (2026).

5.2 Fila da via B e pedido de pedestre

A fila é contada por um CTU com valor predefinido 3, alimentado pela condição `DET_B · ¬E4_VERDE_B`. O contador soma na borda de subida dessa condição: conta cada veículo que chega com B fechada e também o veículo que continua parado sobre o laço quando o verde de B termina, porque a condição volta a 1 nesse instante. A zeragem do contador é a própria etapa de verde B. A demanda é calculada por comparação (`C_FILA.CV ≥ 1`), e a fila cheia é a saída do contador.

```
F2: DET_B · ¬E4_VERDE_B  →  CTU C_FILA (PV = 3)
F3: C_FILA.CV ≥ UM       →  DEM_B
F4: BT_PED · ¬E7_PED · ¬MANUT  →  SET SOL_PED
```

O pedido de pedestre é memorizado em `SOL_PED` e só é apagado quando a fase de pedestre começa; a lâmpada "aguarde" repete essa memória. Como o pedido

é registrado antes das transições, um toque feito depois do verde mínimo encerra o verde A na mesma varredura.

5.3 Transições, etapas e saídas

Cada transição é um bit calculado a partir da etapa de origem, do temporizador da etapa e da condição de saída. Por exemplo, o fim do verde A é escrito como:

```
T12: E1_VERDE_A · T_MIN_A.Q · (SOL_PED + C_FILA.QU + DEM_B ·
T_MAX_A.Q)  →  TR12
S12: TR12  →  RESET E1_VERDE_A, SET E2_AMAR_A
```

As saídas usam o vermelho como estado por padrão: `A_VERMELHO = ¬MANUT · ¬E1 · ¬E2`. Qualquer varredura sem etapa ativa — por exemplo, a de inicialização — mostra vermelho nas duas vias. O vermelho de pedestre fica fixo fora da travessia e pisca na etapa E8, usando o mesmo gerador de intermitência da manutenção.

5.4 Decisões de projeto

Quadro 6 – Principais decisões de projeto

Decisão	Motivo
Fase exclusiva de pedestres	um só grupo focal de pedestres e nenhum conflito entre pedestres e veículos a analisar
Vermelho como estado por padrão	qualquer varredura sem etapa ativa mostra vermelho
Fila contada na borda de `DET_B · ¬E4`	o veículo parado sobre o laço no fim do verde B não é esquecido
Pedido registrado antes das transições	um pedido feito depois do verde mínimo é atendido na mesma varredura
Via B antes do pedestre ao sair do vermelho geral	quem espera em B é servido primeiro, e o pedestre logo depois, sem voltar ao verde A
Manutenção zera as etapas	a volta ao funcionamento sempre passa pelo vermelho geral

Fonte: elaborado pelo autor (2026).

6 RESULTADOS DA SIMULAÇÃO

6.1 Casos de teste

Os 90 casos de teste foram escritos antes da execução e executados no motor do próprio simulador, varredura a varredura (Tabela 1). Todos foram aprovados. Somando os passos de preparo, também conferidos, cada execução do programa tem 4 669 varreduras.

Tabela 1 – Casos de teste por grupo

Grupo	Casos	Varreduras	Resultado
partida e repouso em verde A	4	433	4/4
pedestre: pedido, travessia, botão na travessia, botão preso	43	1 963	43/43
via B: fila, veículo isolado, veículo sobre o laço	30	1 038	30/30
manutenção (amarelo intermitente)	9	107	9/9
parada e retomada da simulação	4	48	4/4
total	90	3 589	90/90

Fonte: elaborada pelo autor (2026), a partir do registro de testes do projeto.

Um interpretador independente executou o mesmo programa e os seis mutantes e coincidiu com o motor do simulador em todos os 32 683 registros de varredura.

6.2 Invariantes e mutantes

Oito invariantes de segurança foram vigiados em 4 668 varreduras com a simulação ligada, sem nenhuma violação (Quadro 7). Os seis mutantes do Quadro 8 foram reprovados pelos casos indicados.

Quadro 7 – Invariantes de segurança vigiados

Id	Propriedade (toda varredura com a simulação ligada)
INV-S1	fora da manutenção, A e B nunca abertos (verde ou amarelo) ao mesmo tempo
INV-S2	pedestre verde só com A e B vermelhos
INV-S3	fora da manutenção, exatamente uma etapa ativa; em manutenção, nenhuma
INV-S4	cada grupo veicular com exatamente uma lâmpada; pedestre nunca verde e vermelho juntos
INV-S5	um verde nunca vai direto a vermelho
INV-S6	um verde só começa com as outras correntes fechadas na varredura anterior
INV-S7	em manutenção, só os amarelos, piscando juntos
INV-S8	"aguarde" igual ao pedido registrado; demanda igual a fila ≥ 1

Fonte: elaborado pelo autor (2026).

Quadro 8 – Mutantes e onde foram detectados

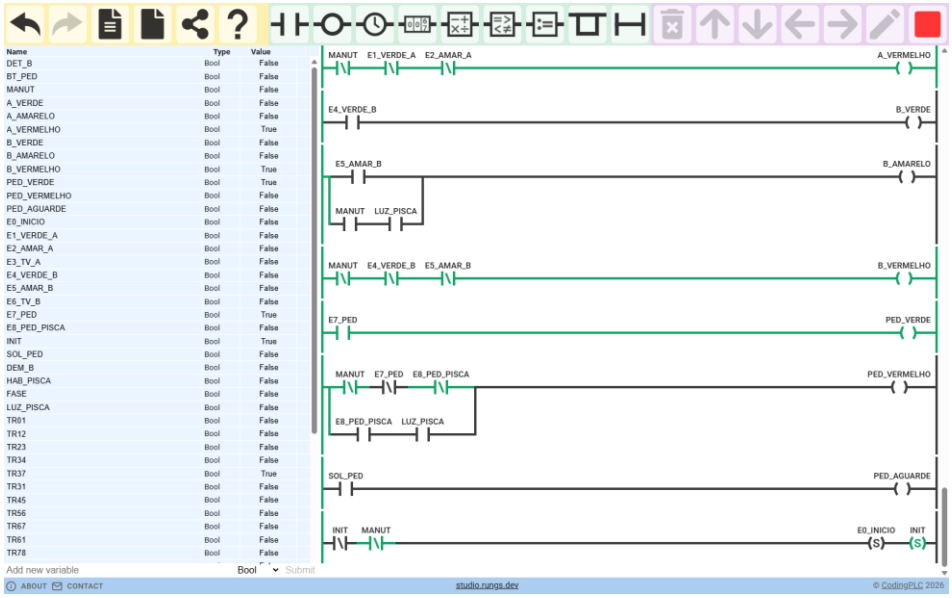
Mutante	Defeito introduzido	Detecção
MUT-S1	amarelo de A escrito em duas bobinas	12 casos; INV-S4 e INV-S5
MUT-S2	transições usam o botão direto, sem memória	21 casos
MUT-S3	fila contada só pela borda do laço	L04–L08: veículo esquecido
MUT-S4	vermelhos sem a condição de manutenção	4 casos; INV-S7
MUT-S5	sem verde mínimo de A	56 casos
MUT-S6	sem verde máximo de A	veículo isolado nunca atendido

Fonte: elaborado pelo autor (2026).

6.3 Estados observados no simulador

As capturas foram tiradas nos estados alcançados pelos mesmos passos dos casos, com o relógio de varredura suspenso durante a foto e conferência de que a tabela exibida é igual ao estado do simulador. A Figura 3 mostra a travessia de pedestres. O link público foi conferido em 12/10/2026, sem sessão: estrutura, comentários dos rungs e valores das tags são idênticos aos do arquivo entregue, assim como o documento gravado no banco do simulador.

Figura 3 – Travessia de pedestres: pedestre verde e veículos vermelhos (caso P07)



Fonte: captura do PLC Simulator Online (CodingPLC, 2026), elaborada pelo autor.

7 DISCUSSÃO DOS RESULTADOS

Os resultados mostram que, nas condições ensaiadas, o programa cumpre a sequência especificada: a via A só deixa o verde quando há demanda ou pedido, a fila de três veículos é atendida no verde mínimo e o veículo isolado no verde máximo, o pedido de pedestre é memorizado e atendido, e a manutenção sempre retorna pelo vermelho geral. A coincidência entre o motor do simulador e o interpretador independente indica que o executor dos testes não distorceu as leituras; a detecção dos seis mutantes indica que os casos são capazes de apontar erros realistas, como a bobina duplicada e o pedido sem memória.

Os testes não demonstram, porém, o comportamento de um cruzamento real. Há limitações importantes:

- a simulação é lógica, sem hardware: o tempo é contado em varreduras de 66 ms e os tempos foram reduzidos para a demonstração;
- a fase exclusiva de pedestres é mais simples de analisar — durante a travessia todos os veículos ficam parados —, mas menos eficiente que travessias em paralelo com o fluxo de veículos;
- não há detecção de lâmpada queimada nem supervisão elétrica; a chave de manutenção representa um comando manual ou um supervisor externo;
- parar e religar a simulação reinicia o tempo da etapa em curso, porque os temporizadores zeram e as etapas não;
- com o botão de pedestre preso, há uma travessia a cada ciclo — o verde mínimo de A, entretanto, é sempre cumprido.

Como melhorias, seriam naturais a contagem de veículos também na via A para ajustar o verde principal, travessias de pedestres em paralelo com o fluxo não conflitante e a supervisão das lâmpadas, com passagem automática ao amarelo intermitente em caso de falha.

8 CONCLUSÃO

O projeto atingiu o objetivo de projetar, implementar e verificar em simulador o controle de um cruzamento semaforizado com pedestres. O GRAFCET de nove etapas, convertido para Ladder com transições calculadas antes da atualização das etapas, produziu um programa organizado em blocos e com cada lâmpada escrita em uma única bobina. Os temporizadores definem as fases e os limites de verde, e o contador transforma a presença de veículos em uma decisão sobre quando servir a via secundária.

A verificação — 90 casos aprovados, 32 683 registros de varredura idênticos entre o simulador e o interpretador, oito invariantes sem violação nas condições testadas e seis mutantes detectados — dá confiança de que o programa faz o que foi especificado nas situações ensaiadas, sem substituir o ensaio em hardware que um sistema real exigiria.

REFERÊNCIAS

CALVETTI, Robson. **CLP e linguagem Ladder**: semana 6. São Paulo: Universidade São Judas Tadeu, 2026a. Material de aula da unidade curricular Sistemas Automatizados.

CALVETTI, Robson. **Fundamentos dos sistemas automatizados**: semana 1. São Paulo: Universidade São Judas Tadeu, 2026b. Material de aula da unidade curricular Sistemas Automatizados.

CALVETTI, Robson. **Processos, sensores e transdutores**: semana 2. São Paulo: Universidade São Judas Tadeu, 2026c. Material de aula da unidade curricular Sistemas Automatizados.

CALVETTI, Robson. **Programação sequencial de CLP**: semana 7. São Paulo: Universidade São Judas Tadeu, 2026d. Material de aula da unidade curricular Sistemas Automatizados.

CALVETTI, Robson. **Projeto para a A3 – Sistemas Automatizados**: desenvolvimento de sistema automatizado utilizando CLP. Versão 1.0. São Paulo: Universidade São Judas Tadeu, 2026e. Especificação do projeto da avaliação A3.

CALVETTI, Robson. **Sistemas sequenciais e GRAFCET/SFC**: semana 5. São Paulo: Universidade São Judas Tadeu, 2026f. Material de aula da unidade curricular Sistemas Automatizados.

CODINGPLC. **PLC Simulator Online**. [S. l.]: CodingPLC, 2026. Simulador de CLP com programação em Ladder. Disponível em: https://app.plcsimulator.online. Acesso em: 12 out. 2026.

JIA, Yue; HARMAN, Mark. An analysis and survey of the development of mutation testing. **IEEE Transactions on Software Engineering**, [S. l.], v. 37, n. 5, p. 649-678, set./out. 2011. DOI: 10.1109/TSE.2010.62.